

BROWSER GAME ENGINE

Технический архитектурный аудит и целевой дизайн

Исходная база: RPG_Engine (MythMill) v1.1.0

Phase 0 — Аудит + Target Architecture + Roadmap миграции

Принцип: READ → UNDERSTAND → MAP → AUDIT → DESIGN → MIGRATE

Полный текст: Browser_Game_Engine_Architecture_Audit_RU.md

А. Краткое резюме

RPG_Engine — JSON-ориентированный текстовый / IF RPG-движок с крупным визуальным редактором. Сильные стороны: квесты v2 (QuestRuntime, questProgress как source of truth), Conditions, Actions, компоненты сцен, контракт Editor.hooks, export, ~173 JS-модуля, baseline квест-тестов 138 passed.

Это НЕ универсальный 2D/3D движок: нет мирового ECS, пайплайна sprite/tilemap/GLTF, физики, изолированного Play Mode. Редактор зависит от глобального Editor, DOM; в истории — рекурсии из-за monkey-patch и багов inspector.

Вердикт

Эволюция в multi-mode Browser Game Engine реальна, если сначала выделить Core Runtime, а Editor сделать его клиентом. Не наращивать 3D патчами editor.html. Переиспользовать QuestRuntime, Conditions, Actions, миграции, no-code UX. Заменить global monkey-patches, DOM-as-runtime и модель «текстовая сцена = весь мир».

В. Текущая архитектура

Слои: Автор → Editor (global + hooks + DOM) → Project JSON → GameEngine + SceneManager + Components + Actions → QuestEventBus → QuestRuntime; параллельно ConditionSystem, Combat, Inventory; ui-renderer (DOM); Save/Load + export.

Вердикты по системам

Система	Вердикт	Комментарий
QuestRuntime / Tasks / Events	KEEP	SoT; покрыто тестами
Conditions / Actions v3	KEEP	Позже namespace
Scene Manager + text scenes	EXTRACT	Стать TextScene
ui-renderer	REFACTOR	Выделить GameUI
Character Creator / Combat	KEEP	Text/Hybrid
Editor.hooks	KEEP	Жёсткий контракт
Global Editor assign	REPLACE	Порядок загрузки
EditorHistory wrap	REFACTOR	Commands
Inspector	REFACTOR	Только registry
Export / bundles	REPLACE	Нормальный dist
2D/3D/Physics/Assets GUID	NEW	Сейчас нет

Инциденты, которые нельзя повторять

Саморекурсия codeFieldIfAdvanced; hooks + поздние Editor.foo=; questStages как SoT; campaign id в generic UI; полный re-render на каждый символ.

С. Целевая архитектура

Browser Game Engine

- └ Engine Core (App, EventBus, Project)
- └ Runtime (Scene, Entity, Component, Input, Audio, Save, Script, Asset Runtime, Game UI)
- └ Rendering (Canvas2D | WebGL2 | WebGPU)
- └ Physics (опционально)
- └ Asset Pipeline (GUID, import, cache)
- └ Editor (не нужен в уже собранной игре)
- └ Export/Build → dist/engine.js + game.js + assets + data

Принципы: Editor не обязателен для запуска build; Game UI ≠ Editor UI; один фундамент Entity/Scene; без monkey-patch; core без DOM; бюджет сложности маленькой команды.

Play Mode

EDIT → снимок → PLAY на клоне runtime → STOP уничтожает runtime → Editor восстановлен. Геймплей не мутирует проект редактора.

Renderer

Абстракция IRenderer. MVP: Canvas2D (2D), WebGL2 (3D). WebGPU — после стабилизации абстракции, не «потому что новее».

D. Карта миграции

KEEP: QuestRuntime, tasks, events, migrate, Conditions, Actions, Character Creator, combat, контракт Editor.hooks, шаблоны TEXT. ADAPTER: текущий scene JSON → TextScene. REFACTOR: ui-renderer, Inspector, History. REPLACE: shell Editor, export pipeline. NEW: ECS core, GUID assets, render backends, physics optional. DELETE после cutover: monolith drift, .bak квестов, мёртвые flag-path прогресса.

F. Четыре режима

Core/Events/ECS/Quests/Conditions/Actions/GameUI — все режимы. Canvas2D/tilemap — 2D (+hybrid). WebGL2/GLTF — 3D (+hybrid). Dialogue/IF/CC — Text (+hybrid). Physics — опционально вне Text. Пространственный viewport — 2D/3D/Hybrid; Text — graph/flow.

G. Дорожная карта (Phase 0-15)

0 Audit — этот отчёт; без массового rewrite.

1 Architecture — пакеты core/runtime/render/editor/build; границы импортов.

2 Core Runtime — EngineApp, EventBus, Entity, Component, Scene; headless-тесты.

3 Assets — GUID-реестр, load by ref.

4 Renderer — IRenderer + Canvas2D; stub WebGL2.

5 Editor Foundation — browser, hierarchy, inspector, viewport; без monkey-patch.

6 Play Mode — изолированный runtime; Stop = снимок.

7 2D — sprite, camera, tilemap; export.

8 3D — GLTF, light, camera; export.

9 Text — adapter сцен/квестов; golden-тесты.

10 Hybrid — composite; диалог поверх мира.

11 Prefab/Anim — Prefab v1; анимации.

12 Build/Export — hashed dist; убрать stale monolith.

13 Testing — ECS, assets, play, quests, boot.

14 Cleanup — adapters/мёртвый код; docs.

15 Stable — engineVersion 2.x; четыре sample.

Каждое изменение: Change → Test → Verify → Commit → Next. Ранний spike Phase 9 снижает риск контента.

I. Реестр рисков

Раздувание scope → phase gates, TEXT+Core первыми. Рекурсии Editor → тесты hooks, запрет monkey-patch. Поломка квестов/save → QuestRuntime + suite миграции. Drift bundle → единый build. DOM UI навсегда → интерфейс Game UI заранее. Nested prefabs/WebGPU рано → отложить. Гигантские PR → маленькие коммиты.

J. Definition of Done

Четыре шаблона; безопасный Play/Stop; dist без Editor; TEXT demo + зелёные квест-тесты; 2D sample движения; 3D GLTF sample; Hybrid с диалогом; ассеты по GUID; нет Runtime→Editor в prod; API Behaviour и Actions задокументированы; риски закрыты или приняты.

Рекомендация: принять Phase 0; Core ECS (1-2) в параллельном дереве; ранний TEXT-adapter; затем пространственные рендереры. Один движок → один Editor → четыре режима → один Runtime → общий фундамент Asset/Scene/Entity. RPG_Engine — база миграции, не потолок.